

Socket Programming for Enabling file transfer from server to client, add file upload functionality from client to server and Implement security features like authentication and encryption.

Key Features

1. File Transfer:

- **Download:** Client requests a file from the server, and the server sends it.
- **Upload:** Client uploads a file to the server, which saves it in a designated directory.

2. Security:

- **Authentication:** Clients provide a valid username and password.
- **Encryption:** All communication (commands, file data) is encrypted using Fernet symmetric encryption.

3. Docker:

- Separate Dockerfiles for server and client.
- Volumes to persist files in server_files and client_files directories.
- Containers communicate over a Docker network.

Prerequisites

- **Docker:** Install Docker Desktop (Windows/Mac) or Docker (Linux).
- **Python:** The code uses Python 3.8+ and the cryptography library.
- **Directory Setup:**
 - Create server_files and client_files directories on the host to persist files.
- **Docker Network:** We'll create a Docker network for communication between containers.

Project Structure

socket_project/

├── server/

```
| |— server.py
| |— Dockerfile
| |— requirements.txt
|— client/
| |— client.py
| |— Dockerfile
| |— requirements.txt
|— server_files/ (create on host)
|— client_files/ (create on host)
```

Server Code

```
import socket
import os
import json
from cryptography.fernet import Fernet
import threading
import hashlib

# Server configuration
HOST = '0.0.0.0' # Listen on all interfaces
PORT = 8080
BUFFER_SIZE = 4096
FILE_DIR = '/app/server_files'
KEY = Fernet.generate_key() # Generate encryption key
CIPHER = Fernet(KEY)

# Simple user database (in production, use a secure database)
USERS = {
    'user1': hashlib.sha256('password1'.encode()).hexdigest(),
    'user2': hashlib.sha256('password2'.encode()).hexdigest()
}
```

```

# Ensure file directory exists

if not os.path.exists(FILE_DIR):
    os.makedirs(FILE_DIR)

def authenticate_client(conn):
    """Authenticate the client with username and password."""
    try:
        # Receive authentication data
        auth_data = conn.recv(BUFFER_SIZE)
        auth_data = CIPHER.decrypt(auth_data).decode()
        auth_dict = json.loads(auth_data)
        username = auth_dict.get('username')
        password = auth_dict.get('password')

        # Verify credentials
        if username in USERS and USERS[username] ==
        hashlib.sha256(password.encode()).hexdigest():
            conn.send(CIPHER.encrypt(b'AUTH_SUCCESS'))
            return True
        else:
            conn.send(CIPHER.encrypt(b'AUTH_FAILED'))
            return False
    except Exception as e:
        print(f"Authentication error: {e}")
        conn.send(CIPHER.encrypt(b'AUTH_FAILED'))
        return False

def handle_client(conn, addr):
    """Handle client requests (download/upload)."""
    print(f"New connection from {addr}")

```

```

# Send encryption key to client
conn.send(KEY)

# Authenticate client
if not authenticate_client(conn):
    print(f"Authentication failed for {addr}")
    conn.close()
    return

while True:
    try:
        # Receive command
        data = conn.recv(BUFFER_SIZE)
        if not data:
            break
        command = CIPHER.decrypt(data).decode()
        command_dict = json.loads(command)

        action = command_dict.get('action')
        filename = command_dict.get('filename')

        if action == 'DOWNLOAD':
            file_path = os.path.join(FILE_DIR, filename)
            if os.path.exists(file_path):
                conn.send(CIPHER.encrypt(b'FILE_EXISTS'))
                with open(file_path, 'rb') as f:
                    while True:
                        bytes_read = f.read(BUFFER_SIZE)
                        if not bytes_read:
                            break

```

```

        encrypted_data = CIPHER.encrypt(bytes_read)
        conn.send(encrypted_data)
        conn.send(CIPHER.encrypt(b'END_OF_FILE'))
        print(f"Sent file {filename} to {addr}")
    else:
        conn.send(CIPHER.encrypt(b'FILE_NOT_FOUND'))
        print(f"File {filename} not found for {addr}")

elif action == 'UPLOAD':
    file_path = os.path.join(FILE_DIR, filename)
    conn.send(CIPHER.encrypt(b'UPLOAD_READY'))
    with open(file_path, 'wb') as f:
        while True:
            data = conn.recv(BUFFER_SIZE)
            if not data:
                break
            decrypted_data = CIPHER.decrypt(data)
            if decrypted_data == b'END_OF_FILE':
                break
            f.write(decrypted_data)
        print(f"Received file {filename} from {addr}")

else:
    conn.send(CIPHER.encrypt(b'INVALID_COMMAND'))
except Exception as e:
    print(f"Error handling client {addr}: {e}")
    break

conn.close()
print(f"Connection closed with {addr}")

```

```
def main():  
    # Create socket  
  
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
    server.bind((HOST, PORT))  
    server.listen(5)  
    print(f"Server listening on {HOST}:{PORT}")  
  
    while True:  
        conn, addr = server.accept()  
        # Handle each client in a separate thread  
        client_thread = threading.Thread(target=handle_client, args=(conn, addr))  
        client_thread.start()  
  
if __name__ == '__main__':  
    main()
```

Server Dockerfile

FROM python:3.8-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY server.py .

CMD ["python", "server.py"]

Server Requirements

cryptography==42.0.5

Client Code

```
import socket

import os

import json

from cryptography.fernet import Fernet


# Client configuration

HOST = 'server' # Docker service name

PORT = 8080

BUFFER_SIZE = 4096

FILE_DIR = '/app/client_files'


# Ensure file directory exists

if not os.path.exists(FILE_DIR):

    os.makedirs(FILE_DIR)


def main():

    # Create socket

    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    client.connect((HOST, PORT))


    # Receive encryption key

    key = client.recv(BUFFER_SIZE)

    cipher = Fernet(key)


    # Authenticate

    username = input("Enter username: ")

    password = input("Enter password: ")

    auth_data = json.dumps({'username': username, 'password': password}).encode()

    client.send(cipher.encrypt(auth_data))
```

```

# Check authentication result

auth_response = client.recv(BUFFER_SIZE)

auth_response = cipher.decrypt(auth_response).decode()

if auth_response != 'AUTH_SUCCESS':

    print("Authentication failed")

    client.close()

    return

print("Authentication successful")


while True:

    print("\nOptions: 1. Download file 2. Upload file 3. Exit")

    choice = input("Enter choice (1-3): ")

    if choice == '1':

        filename = input("Enter filename to download: ")

        command = json.dumps({'action': 'DOWNLOAD', 'filename': filename}).encode()

        client.send(cipher.encrypt(command))

        # Check if file exists

        response = client.recv(BUFFER_SIZE)

        response = cipher.decrypt(response).decode()

        if response == 'FILE_NOT_FOUND':

            print(f"File {filename} not found on server")

            continue

        # Receive file

        file_path = os.path.join(FILE_DIR, filename)

        with open(file_path, 'wb') as f:

            while True:

```



```

        data = client.recv(BUFFER_SIZE)

        if not data:
            break

        decrypted_data = cipher.decrypt(data)

        if decrypted_data == b'END_OF_FILE':
            break

        f.write(decrypted_data)

    print(f"Downloaded file {filename}")

elif choice == '2':
    filename = input("Enter filename to upload: ")
    file_path = os.path.join(FILE_DIR, filename)
    if not os.path.exists(file_path):
        print(f"File {filename} not found locally")
        continue

    command = json.dumps({'action': 'UPLOAD', 'filename': filename}).encode()
    client.send(cipher.encrypt(command))

    # Wait for server readiness
    response = client.recv(BUFFER_SIZE)
    response = cipher.decrypt(response).decode()
    if response != 'UPLOAD_READY':
        print("Server not ready for upload")
        continue

    # Send file
    with open(file_path, 'rb') as f:
        while True:
            bytes_read = f.read(BUFFER_SIZE)
            if not bytes_read:

```

```

        break

    encrypted_data = cipher.encrypt(bytes_read)

    client.send(encrypted_data)

    client.send(cipher.encrypt(b'END_OF_FILE'))

    print(f"Uploaded file {filename}")

elif choice == '3':

    break

else:

    print("Invalid choice")

client.close()

print("Disconnected from server")

if __name__ == '__main__':

    main()

```

Client Dockerfile

FROM python:3.8-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY client.py .

CMD ["python", "client.py"]

Client Requirements

cryptography==42.0.5

Create Directories:

- On your host machine, create:

```
mkdir socket_project
```

```
cd socket_project
```

```
mkdir server client server_files client_files
```

- Place server.py, Dockerfile, and requirements.txt in socket_project/server/.
- Place client.py, Dockerfile, and requirements.txt in socket_project/client/.

Create Docker Network:

- Create a bridge network for communication:

```
docker network create socket-net
```

Build Server Image:

- Navigate to socket_project/server/:

```
cd server
```

```
docker build -t socket-server .
```

Build Client Image:

- Navigate to socket_project/client/:

```
cd ../client
```

```
docker build -t socket-client .
```

Run Server Container:

- Run the server with a volume to persist files:

```
docker run -d --name server --network socket-net -p 8080:8080 -v  
$(pwd)/../server_files:/app/server_files socket-server
```

Run Client Container:

- Run the client with a volume for file storage:

```
docker run -it --name client --network socket-net -v $(pwd)/../client_files:/app/client_files socket-client
```

- The -it flag allows interactive input for username/password and menu choices.

Interact:

- In the client container, enter username (user1 or user2) and password (password1 or password2).
- Choose options to download or upload files:
 - For download, enter a file name from server_files (e.g., place test.txt in server_files on the host).
 - For upload, place a file in client_files on the host and enter its name.

Stop Containers:

- Stop and remove containers when done:

```
docker stop server client
```

```
docker rm server client
```

Security Features

1. Authentication:

- The server uses a dictionary of usernames and SHA-256 hashed passwords.
- Passwords are hashed before comparison to avoid storing plaintext.
- In production, use a secure database and stronger hashing (e.g., bcrypt).

2. Encryption:

- Fernet (AES-128 in CBC mode with PKCS7 padding and HMAC) encrypts all communication.
- The server generates a key and sends it to the client at connection start.
- Note: The key is sent in plaintext. In production, use SSL/TLS for secure key exchange.

Notes

- **Docker:**

- The server listens on 0.0.0.0 to accept connections from the Docker network.
- The client connects to the server using the service name server (resolved via Docker network).
- Volumes ensure files persist on the host (server_files and client_files).

- **File Transfer:**

- Files are transferred in chunks (BUFFER_SIZE = 4096) for efficiency.
- The END_OF_FILE marker signals the end of file transmission.
- **Threading:**
 - The server uses threading to handle multiple clients concurrently.
- **Security Limitations:**
 - The encryption key is sent in plaintext. Use SSL/TLS in production (e.g., Python's ssl module).
 - The user database is a simple dictionary. Use a secure database in production.
- **Error Handling:**
 - Handles file not found, network errors, and invalid commands.

Example Usage

1. Place test.txt in socket_project/server_files/ on the host.
2. Build and run the server and client containers as described.
3. In the client container, authenticate with user1/password1.
4. Choose option 1, enter test.txt to download it to client_files/.
5. Choose option 2, enter a file from client_files/ to upload to server_files/.

Troubleshooting

- **Connection Issues:** Ensure the Docker network (socket-net) is created and both containers are on it.
- **File Access:** Verify that server_files and client_files directories exist on the host and are writable.
- **Port Conflicts:** If port 8080 is in use, change the port in server.py, client.py, and the docker run command.

Let me know if you need help with Docker setup, SSL/TLS integration, or additional features like multi-client support!